

第4章

中断系统与GPIO

第4章 MSP430单片机中断系统与输入输出端口



- 4.1 中断系统
- 4.2 数字I/O端口
- 4.3 端口应用

4.3 端口应用



- 4.3.1 普通I/O端口
- 4.3.2 外部中断



4.3.1 普通I/O端口

为减轻程序员的负担，MSP430的寄存器均在相应的头文件中做了宏定义，同时还对一些常量和函数做了宏定义。

如：
`P2DIR = 0xF0;` //表示P2端口高4位为输出，低4位为输入
`tmp = P1IN;` //读取端口P1的值给变量tmp



4.3.1 普通I/O端口

MSP430不支持位操作和位变量。C430的位操作是通过变量与掩膜位间的逻辑操作实现的。

一般将感兴趣的位设为1（即掩膜位），其他为0。

如：预将R5的0、1位设为1，
$$R5 \mid = 0x03$$



4.3.1 普通I/O端口

为方便，在头文件中对常见的掩膜数进行了宏定义，记为**BITx**。

常量	数值	常量	数值	常量	数值	常量	数值
BIT0	0x0001	BIT4	0x0010	BIT8	0x0100	BITC	0x1000
BIT1	0x0002	BIT5	0x0020	BIT9	0x0200	BITD	0x2000
BIT2	0x0004	BIT6	0x0040	BITA	0x0400	BITE	0x4000
BIT3	0x0008	BIT7	0x0080	BITB	0x0800	BITF	0x8000

R9 |= BIT0 + BIT7; //对R9.0和R9.7置1

R8 &= ~BIT7; //对R8.7清零

R10 ^= BIT7 + BIT5; //对R10.7和R10.5取反



4.3.1 普通I/O端口

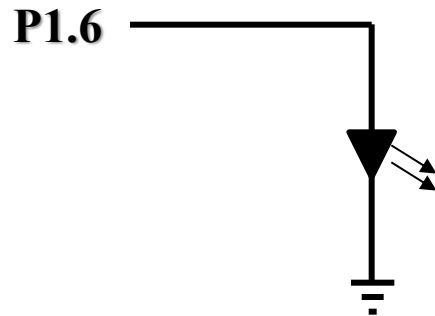
在使用I/O口时：

- 1).进行功能选择(PxSEI & PxSEL2)
- 2).根据应用情况设置数据的传输方向(PxDIR)
 - ◆若是数据输出，则需要将数值写到PxOUT中
 - ◆若是数据输入，则到PxIN读取引脚的值
- 3).是否需要上拉/下拉电阻(PxREN)



4.3.1 普通I/O端口

- 例1 试编写C语言程序，使P1.6引脚处的发光二极管不停地闪烁。连接方式见下图





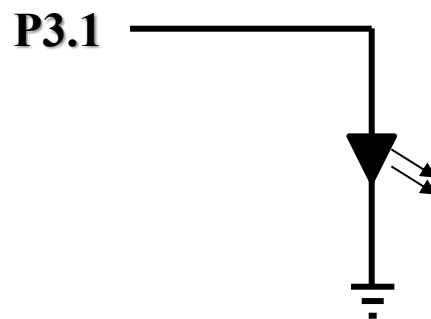
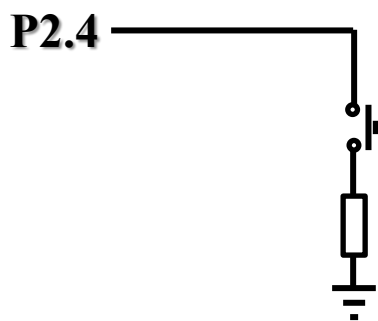
4.3.1 普通I/O端口

```
void main(void)
{
    unsigned int i;
    WDTCTL = WDTHOLD + WDTPW;           //关闭看门狗
    P1DIR = BIT6;                       //使能P1.6为输出
    while(1)
    {
        P1OUT ^= BIT6;                 //循环
        for(i=0; i<5000; i++);         //对输出置反
    }                                   //延时
}
```



4.3.1 普通I/O端口

练习 试编写C语言程序，测试P2.4引脚处的按键是否被按下，若按下，则使P3.1处的发光二极管点亮。否则发光二极管不亮。



4.3 端口应用



- 4.3.1 普通I/O端口
- 4.3.2 外部中断



4.3.2 外部中断

- P1和P2端口的任一引脚均具有中断功能，即具有16个外部输入可屏蔽中断源
- P1端口的8个引脚中断源共享同一个中断向量
- P1中断的配置通过编程P1IE，P1IFG和P1IES三个寄存器实现(尚需set GIE)
- P2端口与P1端口相同



4.3.2 外部中断

- 总中断控制位GIE位于SR中，无法直接操作，因此需要通过特殊的函数实现总中断的开关。

(1) **void __disable_interrupt(void):** 该函数的功能是关闭总中断。简写为 **_DINT()**。

(2) **void __enable_interrupt(void):** 该函数的功能是开启总中断。简写为 **_EINT()**。



4.3.2 外部中断

```
void main(void)
```

```
{
```

```
.....
```

```
P2IES = BIT0 + BIT1 + ...+BIT7; // 选择下降沿触发中断
```

```
P2IE = BIT0 + BIT1 + ...+BIT7; // 中断使能;
```

```
P2IFG = 0; // 清标志位;
```

```
_EINT(); // 开总中断;
```

```
.....
```

```
}
```

```
#pragma vector = PORT2_VECTOR
```

```
__interrupt void PORT2_ISR(void)
```

```
{
```

```
    if(P2IFG & BIT0)
```

```
    {
```

```
        功能代码 0; // 处理中断源来自 P2.0 处的代码
```

```
    }
```

```
    .....
```

```
    P2IFG = 0; // 必须清除中断标志位
```

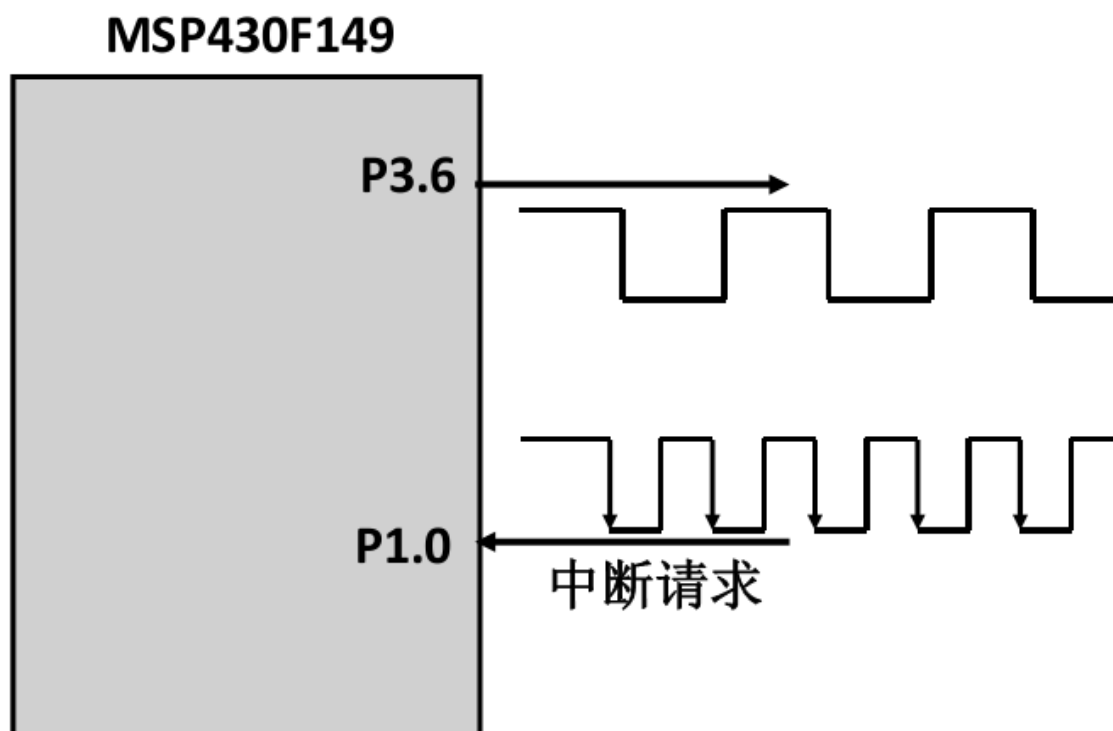
```
}
```



4.3.2 外部中断

例： P1.0中断编程

以中断方式响应P1.0上的下降沿，
每来一个下降沿，使 P3.6 的输出翻转一次





4.3.2 外部中断

● 主程序

```
void main(void)  
{ // IO设置  
    P3DIR |= 0x40;  
    P1IE |= 0x01;  
    P1IES |= 0x01;  
    _EINT();  
    while(1);  
}
```

● 中断子程序

```
#pragma vector =  
PORT1_VECTOR  
__interrupt void  
PORT1_ISR(void)  
{ // IO设置  
    P3OUT ^= 0x40;  
    P1IFG &=~ 0x01;  
}
```



4.3.2 外部中断

- 中断函数既没有输入参数也没有输出参数且不能被其它函数调用。
- 通常中断函数体内定义的变量均是局部变量，其生存周期只在函数体有效。
- 中断函数执行完后所有局部变量的值将丢失以致无法保留。



4.3.2 外部中断

- 但在设计诸如中断计数的程序时，通常需要将中断函数中某些变量的值保留下来，即中断函数返回后变量值不丢失。
- 若要保留这些变量值，可使用两种方法：



4.3.2 外部中断

- ◆ 一种通常使用的方法是定义全局变量。由于全局变量在整个程序里都是有效的，所以中断函数中也可以使用。并且当中断函数结束后，当前值将被保留，以供下次或其他函数使用。但要控制程序中全局变量的个数以防RAM不够用。
- ◆ 另一种方法是在中断函数体内声明变量时，前面加上关键词static，是说明该变量为本地全局变量，也可将当前值保留下来。



4.3.2 外部中断

练习： MSP-EXP430G2开发板上P1.3接了一个按键，P1.0和P1.6接了LED（用跳线帽连接），编写一段代码实现：两个LED保持1亮1灭，每次按下P1.3后，LED交换亮灭状态，要求不阻塞CPU。

